

Arkitekturdokument

Grupp 13

2026-03-30

Version 0.2



Status

Granskad	Szilvia Varro-Gyapay	2026-xx-xx
Godkänd	Szilvia Varro-Gyapay	2026-xx-xx

Projektidentitet

Grupp E-post: tddd96_2026vt_ax_13@groups.liu.se

Beställare: Johan Munck, Börjes Koncernen
Tfn: +46733318552
E-post: johan@borjeskoncernen.se

Handledare: Szilvia Varro-Gyapay
Tfn: +4613284092
E-post: szilvia.varro-gyapay@liu.se

Kursansvarig: Lena Buffoni
Tfn: +4613284046
E-post: lena.buffoni@liu.se

Projektdeltagare

Namn	Ansvar	E-post
Karl Lennartsson	Teamledare	karle221@student.liu.se
Simon Andersson	Dokumentansvarig	siman020@student.liu.se
Elias Facq	Analysansvarig	eliwa076@student.liu.se
Jacob Ahling Hult	Utvecklingsledare	jacah067@student.liu.se
Jakob Renqvist	Arkitekt	jakre732@student.liu.se
Viktor Wallberg	Kvalitetssamordnare	vikwa448@student.liu.se
Felix Hallström	Testledare	felha291@student.liu.se

INNEHÅLL

1	Inledning	1
2	Systemarkitektur	1
2.1	Vår databas	1
2.1.1	Design	1
2.1.2	Supabase	4
2.2	Full-stack utveckling med Next.js	5
2.2.1	Kommunikation mellan <i>backend</i> och <i>frontend</i>	6
2.3	Funktioner för <i>Backend</i>	6
2.4	Funktioner för <i>Frontend</i>	6
2.4.1	Inloggningsskärm	6
2.4.2	Översikt	7
2.4.3	Simulator	7
2.4.4	Adminpanel	8
2.4.5	Konto	8
2.4.6	Inställningar	8
2.4.7	Notifikationer	8
2.5	API och iLog	9
3	Systemanatomi	9

DOKUMENTHISTORIK

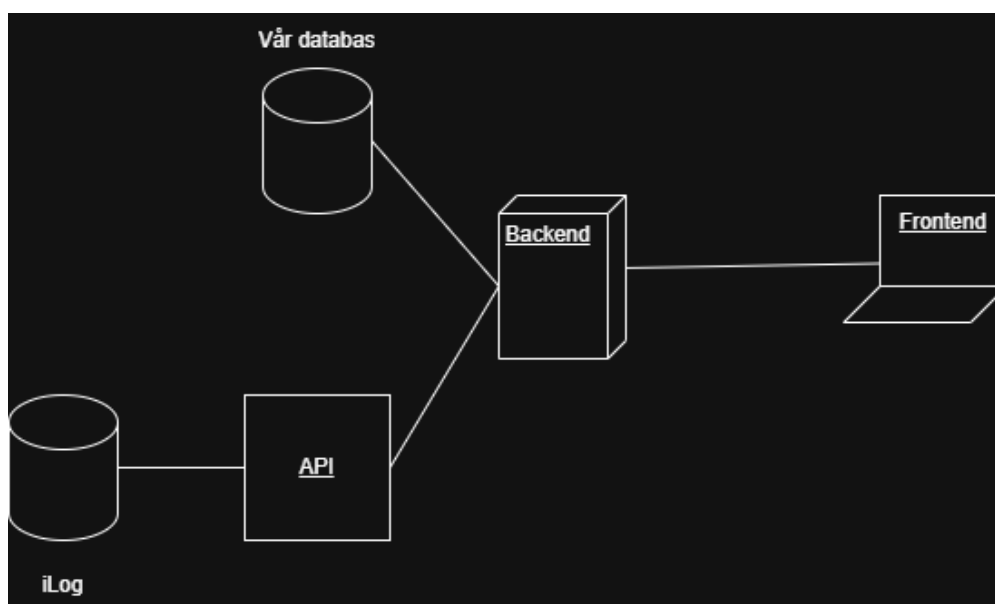
Version	Datum	Utförda förändringar	Utförda av	Granskad
0.1	2026-02-16	Första utkast	Jakob Renqvist	2026-02-18
0.2	2026-03-26	Första revision	Jakob Renqvist	2026-03-26

1 INLEDNING

Detta dokument beskriver en överliggande arkitektur för att utveckla ett system för Börjes Koncernen. Systemet skall underlätta för trafikledare att göra avgörande beslut inom logistiken hos kundföretag. Detta genom att ta hänsyn till historisk data och kostnader för frakter för att beräkna ett estimerat pris för en framtida frakt.

2 SYSTEMARKITEKTUR

I detta kapitel diskuteras den generella arkitekturen över systemet detta projekt skall utveckla. I figur 1 visas ett diagram över systemarkitekturen. De system detta projekt kommer fokusera på är *Vår databas*, *Backend* samt *Frontend* enligt figur. De resterande komponenterna (*API* och *iLog*) är redan utvecklade och färdigställda av kunden. I följande underkapitel diskuteras samtliga komponenter och ramverk som används i detalj.



Figur 1: Systemarkitektur över alla komponenter. Detta är version 1 utgivet 5 feb 2026.

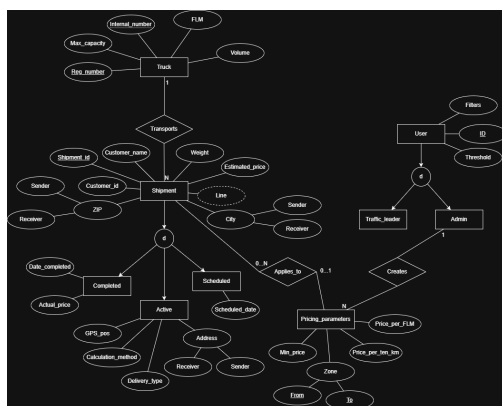
2.1 Vår databas

I detta kapitel förklaras hur designen av databasen gjordes samt hur denna design implementeras i Supabase

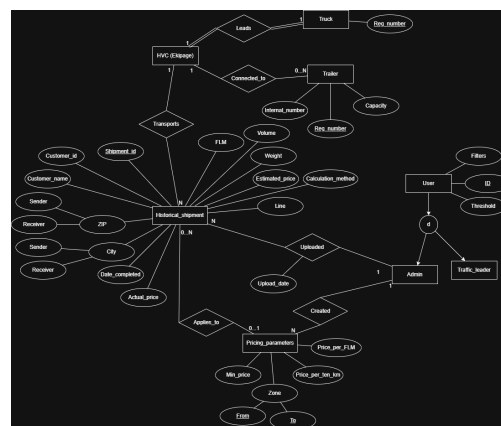
2.1.1 Design

Den data i databasen detta projekt kommer utveckla har justerats och förändrats en hel del fram till dagens datum (2026-03-25). Därför presenteras tre versioner av ett *Enhanced Entity-Relationship model* (kallat EER-diagram i denna rapport) i figur 2a, figur 2b och figur 2c. Ett EER-diagram är ett välkänt och standardiserat arbetsverktyg för att

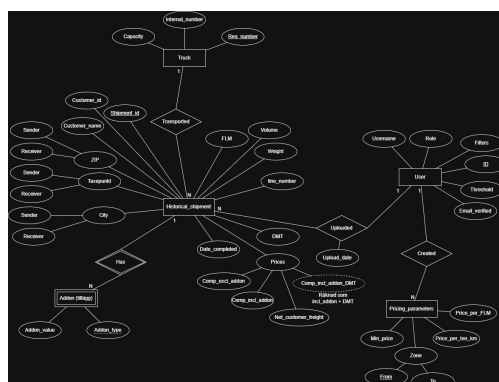
utveckla databaser. Att sedan översätta detta diagram till en faktisk relationsdatabas är också standardiserat. Efter översättning och mer lärdöm om hur Supabase hanterar databaser så utvecklades den senaste versionen, version 7 i figur 2c. Vidare småförändringar kommer troligtvis ske, men i dessa fall kommer inte någon större bearbetning på EER diagrammet göras. Dessa förändringar kommer enbart ske i Supabase. Notationen och vad allt betyder i ett EER-diagram kommer ej förklaras i denna rapport (hade varit ganska lång).



(a) Version 1



(b) Version 3



(c) Version 7

Figure 2: Tre utkast på EER-modellen över projektets databas.

I figur 2a visas ett första utkast på EER-diagrammet. I detta diagram är det tänkt att vi ska spara lastbilar (*Truck* i diagram) med attributer som registreringsskylt osv. En (1) lastbil ska kunna transportera flera (N) frakter (*Shipment* i diagram) vilket ger en 1:N relation. En frakt ska också specialiseras disjunkt till klar (*Completed*), aktiv (*Active*) och schemalagd (*Scheduled*). Vidare ska databasen också spara användare som också specialiseras disjunkt till trafikledare (*Traffic_leader*) och administratör (*Admin*). Det en trafikledare ska kunna göra i systemet har inget med databasen att göra, så den enheten har inga attribut eller relationer. En administratör ska kunna skapa en generell prisparameter som sista steg i trappstegsmodellen som i sin tur kan eller inte (ty 0...1 och 0..N restriktionerna) ha en relation med en frakt.

I figur 2b visas version 3 av EER-diagrammet. Två stora förändringar har skett i samband med ett flertal mindre förändringar. En av de stora förändringarna är att enheten *Truck* har delats upp mer specifikt så vi har lastbildshuvudet som kan kopplas till ett ekipage samt N antal släp (*Trailer*) som också kan kopplas till samma ekipage. Ett ekipage har sedan en relation med en historisk frakt (*Historical_shipment*) som är del av den andra stora förändringen. I version 1 var en frakt delad i tre möjliga specialiseringar. Men detta kan vara onödigt då system som hanterar schemalagda frakter och aktiva frakter redan hanteras i kundens existerande system. Dessa två specialiseringar kan då tas bort vilket hade gjort att en frakt endast kan vara historiskt. Därav har namngivningen för en frakt (*Shipment* i v1) ändrats till *Historical_shipment* i v3. Attributerna för *Completed*-frakter har lagts till i *Historical_shipment* medan attributerna för de andra specialiserade enheterna har tagits bort.

Den sista figuren, figur 2c, visar version 7 och troligtvis den sista upplagan av detta diagram. Mest märkvärdigt i jämförelse med version 3 har attributen *Price* nyanserats, relationen *Has* med tillhörande enhet *Addon* (*Tillägg*) har lagts till samt att enheten *User* har fått några nya attributer och relationen *Applies_to* har tagits bort.

De mindre förändringarna kommer ej diskuteras i större omfattning, men här är en log på alla förändringar mellan samtliga versioner:

Version 1 (2026-02-05)

Första utgåva. Inga möjliga förändringar från tidigare.

Version 2 (2026-02-09)

- Flyttade attributen *FLM* och *Volume* från *Truck* till *Shipment*.
- Döpte om attributen *Max_capacity* i *Truck* till *Capacity*.
- Flyttade attributen *Calculation_method* från *Active* till *Shipment*.
- Ändrade attributen *Line* till att inte vara derivat.
- Lade till relationen *Uploaded* mellan *Admin* och *Completed*.

Version 3 (2026-02-12)

- Omstrukturerade *Truck* till tre separata enheter.
- Tog bort specialiseringen av *Shipment* till *Active* och *Scheduled*.
- Döpte om *Shipment* till *Historical_shipment*.
- Lade till attributen *Date_completed* och *Actual_price* i *Historical_shipment*.
- Kopplade om relationen *Uploaded* till *Historical_shipment*.

Version 4 (2026-02-22)

- Bytte tillbaka `Truck` till att vara en enhet
- Döpte om `Transports` till `Transported`
- Kryssade för attributerna `estimated_price` och `calculation_method`. Behöver troligtvis inte sparas i databas
- Adderade attribut `DMT` till `historical_shipment`
Adderade enhet `Addon` med relation till `historical_shipment` som kan lagra tillägg till sändningar.

Version 5 (2026-02-26)

- Tog bort `estimated_price` och `calculation_method` som tidigare var kryssade
- Gjorde om attribut `prices` till multivärderad med fler attributer om priser
- Lade till attributer `Username` och `Password` på enhet `User`

Version 6 (2026-03-04)

- Tog bort relation `Applies_to`

Version 7 (2026-03-25)

- Tog bort attribut `Password` från enhet `User`
- Lade till enheter `Role` och `Email_verified` på enhet `User`
- Tog bort specialisering av `User` Till `Admin/Traffic_leader`

2.1.2 Supabase

För att implementera denna design används Supabase. Supabase är ett *Backend-as-a-Service* (BaaS) system med öppen källkod som bygger på databashanteringssystemet PostgreSQL. Supabase hjälper med många vanliga funktioner man vill ha i ett system som en kraftfull databas, automatisk och högteknologisk kryptering, fillagring samt tillgång till realtidsdata. Dessutom sköter Supabase de vanligaste funktionerna inom autentisering och *token*-behandling (validering, förnying osv) genom att registrera användare, logga in användare och logga ut användare av sig själv. Deras kollaboration och sammanvävning med ramverket Next.js (som diskuteras i kapitel 2.2) gör detta ännu enklare.

Som nämnt innan bygger Supabase på databashanteringssystemet PostgreSQL. Detta *Object-relational Database Management System* (ORDBMS) är också välkänt för sin säkerhet, pålitlighet, avancerade funktioner samt strikt dataintegritet. Med tanke på att kunden har höga krav på just datasäkerhet är PostgreSQL ett perfekt val för databashanteringssystem i detta projekt. PostgreSQL har också en hel del andra fördelar som hjälper med utveckling av en databas som *triggers*, *policy* och *RLS*-säkerhet. Men dessa kommer inte förklaras i större del i denna rapport (ganska lång förklaring) så gör gärna forskning själv på dessa områden.

Valet av Supabase som databashanteringssystem kom som rekommendation av kunden. Inget alternativ till Supabase diskuterades då samtliga i gruppen tyckte att systemet verkade perfekt för kundens önskemål samt utmanande och utbildande för gruppens medlemmar. Att jobba med ett modernt system som Supabase hade ingen i gruppen någon större erfarenhet med.

2.2 Full-stack utveckling med Next.js

Projektet bygger på ramverket Next.js som i sin tur bygger på ramverket React. Next.js var utvecklat av företaget Vercel år 2016 och är idag ett kraftfullt verktyg för webbutveckling med en pseudoliknande full-stack arkitektur. Ramverket är byggt för att hantera båda programmeringsspråken Javascript och Typescript, men det är rekommenderat att använda Typescript. I detta projekt använder vi Typescript.

Valet av Next.js/Vercel tog ett par iterationer och möten med kund. Kunden presenterade två alternativ: Vercel och Lovable. Inom gruppen var det långa diskussioner i vilket utvecklingsmiljö vi skulle använda. Fördelar och nackdelar med båda system togs fram, men i slutändan var samtycket i gruppen att Lovables starka LLM och AI-integration inte skulle ge oss mycket utbildning. Resultatet blev att vi valde Next.js vilket kunden också accepterade.

I ett Next.js projekt är både *backend*- och *frontend*-delarna utvecklade i samma miljö. Traditionellt sätt är det vanligt att skapa dessa separat i två olika system och ha ett API för att möjliggöra kommunikation mellan lagerna. Medan Next.js fortfarande stödjer samma struktur med en *backend* som körs på en server och en *frontend* som körs hos klienten, så utvecklas dessa i samma utvecklingsmiljö.

Denna speciella struktur har som fördel att båda lagerna kan sättas i drift tillsammans via Vercel, som själva hanterar vilka delar som är backend respektive frontend. Den största nackdelen med Next.js är att denna typ av strukturering är unik till just Next.js så kan det ta tid att lära sig hur ramverket fungerar.

Utöver detta har Next.js många fler fördelar som *Server-side rendering*, *static generation* och hybridgenerering. Medan dessa fördelar är bra så påverkar dessa främst webbsidor som vill visas bland toppresultaten av en sökmotor som Google, Bing osv. Detta är perfekt för en blogg, affärshemsida eller liknande, men eftersom detta projekt ska användas som ett internt system hos kunden så är dessa fördelar inte så applicerbara. Dock har Next.js också fördelen för högt interaktiva system i samband med React som Next.js bygger på. Denna fördel kommer detta projekt dra stor nytta av.

Rent programmatiskt innebär detta att samtliga undermappar och underfiler till `app/api` (alternativt `pages/api` för äldre system) är *backend*-kod, och allt annat under `app/` mappen är kod för *frontend*.

Nedan är ett exempel på hur filstrukturen kan se ut och vilka rutter de innebär. Filer som slutar på `.tsx` är generellt filer som returnerar HTML-kod som används i frontend, och filer som slutar på `.ts` är *backend*-filer eller andra hjälpfiler i Typescript. Notera att `[url]` nedan kan vara `localhost:3000` eller den URL som webbsidan ligger på beroende på om åtkomst till systemet sker lokalt eller produktionsmässigt.

```
app /
+-- page . tsx          -> [ url ]/
+-- login /
|   +-- page . tsx      -> [ url ]/ login
```

```
|  '-- test /  
|      '-- page.tsx    -> [url]/login/test  
|-- api /  
    '-- users /  
        '-- route.ts    -> [url]/api/users
```

2.2.1 Kommunikation mellan *backend* och *frontend*

Next.js har två primära sätt att kontrollera hur backend och frontend kommunicerar med varandra: *App router* och *Pages router*. Detta projekt använder en *App router* då denna teknik är nyare och rekommenderad av Vercel. *Pages router* är ett äldre system och bibehålls endast för bakåtkompatibilitet.

Ett projekt som använder en *App router* har många fördelar över dess konkurrent. Exempelvis ger en *App router* upphov till olika exponerbara funktioner för individuella HTTP-metoder (GET, POST, PUSH, DELETE osv) medan en *Pages router* enbart exporterar en funktion som måste hantera alla HTTP-metoder. En *App router* ger också möjlighet till individuella design och layouter genom ett nästlat layout-system. Alltså att en `layout.ts`-fil i varje mapp definierar designen för just den ruten. Om en layout-fil inte finns i en viss mapp så ärvs den layout-filen från föräldramappen.

2.3 Funktioner för *Backend*

På *Backend*-sidan av projektet kommer den huvudsakliga kalkyleringen av ett estimerat pris att ske. Denna estimering kommer ske genom att gå igenom en *trappstegsmodell* som försöker uppfylla vissa kriterier. Om ett visst kriterium har uppfyllts kommer ett estimerat pris räknas ut baserat på stora mängder historisk data som finns i vår databas. Se tabell 1 för hur trappstegsmodellen ser ut.

Utöver detta kommer *backend*-delen av projektet också hantera kommunikation med iLog via deras API. På så sätt behöver *frontend*-delen av projektet endast oroa sig över ett API. Mer information om detta finns under kapitel 2.5. Dessutom ska rutter för generell data som användare, admindata och annat intressant i databasen också exponeras genom API:et vår *backend* kommer exponera.

2.4 Funktioner för *Frontend*

Idag har hemsidan ett par vyer som trafikledare och administratörer kommer att interagera med. Dessa är främst: Inloggningsskärm, Översikt, Simulator, Adminpanel, Konto, Inställningar, Notifikationer samt en logga ut knapp. Nedan förklaras varje komponent samt deras syfte kort.

2.4.1 Inloggningsskärm

När en trafikledare navigerar till vår hemsida kommer de mötas av en inloggningsskärm. Idag är det oklart om Next.js har stöd för automatisk inloggning via *persistent cookies* som man ofta ser i nya webbsidor. Här är det viktigt att denna skärm endast är menad för att **logga in** och inte kunna registrera ett nytt konto. Att registrera ett nytt konto är något som endast en administratör ska kunna göra.

Tabell 1: Trappstesmodell för prisestimering

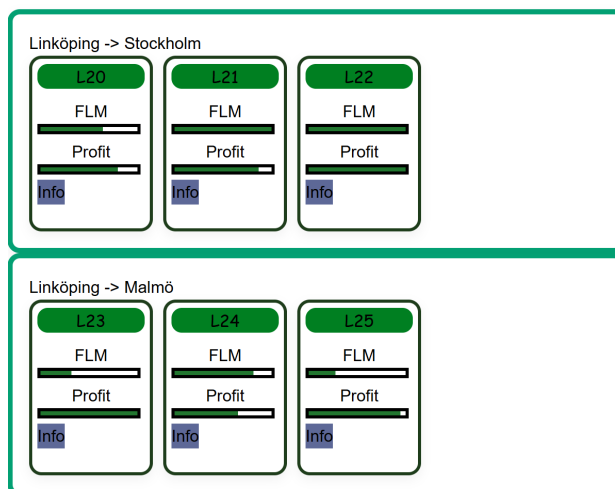
Steg	Kriterium	Beskrivning
1	Kundnamn + taxepunktsrelation + viktklass	Ger mest specifik prisestimering. En träff här innebär att exakt samma kund tidigare skickat gods av liknande vikt på samma rutt
2	Kundnamn + viktklass	Matchar på kundnamn och viktklass oavsett rutt. Används när kundspecifika historisk data finns men inte för den specifika taxepunktsrelationen.
3	Kundnamn	Matchar enbart på kundnamn. Använder ett genomsnitt av kundens historiska prisnivå, skalat med en förhållandekoefficient mot de generella Sverigesnittet.
4	Taxepunktsrelation + viktklass + avstånd	Steget lämnar kundspecifika data och matchar i stället på taxepunktsrelation, viktklass och avståndsintervall. Representativt pris hämtas ur en tabell över genomsnittliga priser för körningar med liknande rutt och vikt.
5	Linje + viktklass + avstånd	Matchar på trafiklinjen (kombinationen av avsändare och mottagare), viktklassen och avståndet. Linjetabellen ger ett genomsnittspris för körningar på samma logistiklinje.
6	Avstånd + viktklass	Det minst specifika steget och den sista utvägen. Estimeringen baseras uteslutande på det körda avståndet och viktklassen, utan hänsyn till kund eller rutt.

2.4.2 Översikt

Detta är huvudvyn i webbsidan. Här kommer små "kort" presenteras för trafikledare där dem kan se information om en viss frakt. I figur 3 finns en tidig skiss på två av dessa kort. Vi valde att efterlikna en design som trafikledare redan är vana vid. I deras existerande system iLog finns liknande kort med sammanhängande linjer för varje rutt. På varje av de tre linjerna i varje kort kan trafikledare se info om samtliga linjers flakmeter och lönsamhet med data hämtat från iLogs API och vår prisestimering. En liten infoknapp kommer ge möjligheten att visa mer information om just denna frakt. Här är det alltså tanken att trafikledare kommer spendera mest tid när dem använder vårt system.

2.4.3 Simulator

I denna vy kommer trafikledare kunna mata in parametrar fritt, alternativt utgå från en viss frakt och ändra dess parametrar, för att se hur vår prisestimering kommer förändras. Detta kommer göra att trafikledare kommer kunna experimentera med andra uppsättningar av parametrar för en viss frakt och se vad som blir mest lönsamt.



Figur 3: Exempel på två kort som visar olika linjer och dess frakters lönsamhet

2.4.4 Adminpanel

Här ska endast administratörer ha tillgång. I denna vy ska en admin kunna se en filtrerbar lista över de trafikledare som använder systemet och vilka områden dem har ansvar över. Man ska också kunna klicka på ett namn i denna lista för att se mer information om den trafikledaren. Admin ska också kunna skapa nya användare genom att ange deras email och ett lösenord dem vill använda. Dessutom ska admin kunna skicka meddelanden till alla användare av systemet som visas under notifikationer-vyn som beskrivs i kapitel 2.4.7. Men det huvudsakliga en admin ska göra i denna panel är att ladda upp en stor CSV-fil på historisk data som ska laddas in i vår databas. På detta sätt kan databasen hållas uppdaterad genom periodisk uppdatering med ny data.

2.4.5 Konto

Här ska en användare kunna se sitt konto och intressant data kopplat till dem som område, användartyp, email osv. Ska också kunna ange ett tröskelvärde som kommer användas som grund till visualisering av korten på översiktsmenyn. Möjligtvis ha en funktion som kan uppdatera email som är kopplat till detta konto, alternativt skicka en förfrågan till en admin på att en trafikledare vill byta lösenord.

2.4.6 Inställningar

Här ska en användare kunna ändra på inställningar som ljus/mörkt tema, område samt kunna återställa lösenord genom att traditionellt sätt ange nuvarande lösenord och det nya lösenordet.

2.4.7 Notifikationer

Här kommer meddelanden från administratörer dyka upp. Möjligt också att ett messenger-system mellan trafikledare också kan införas, men endast i den mån om detta anses vara nyttigt enligt kund.

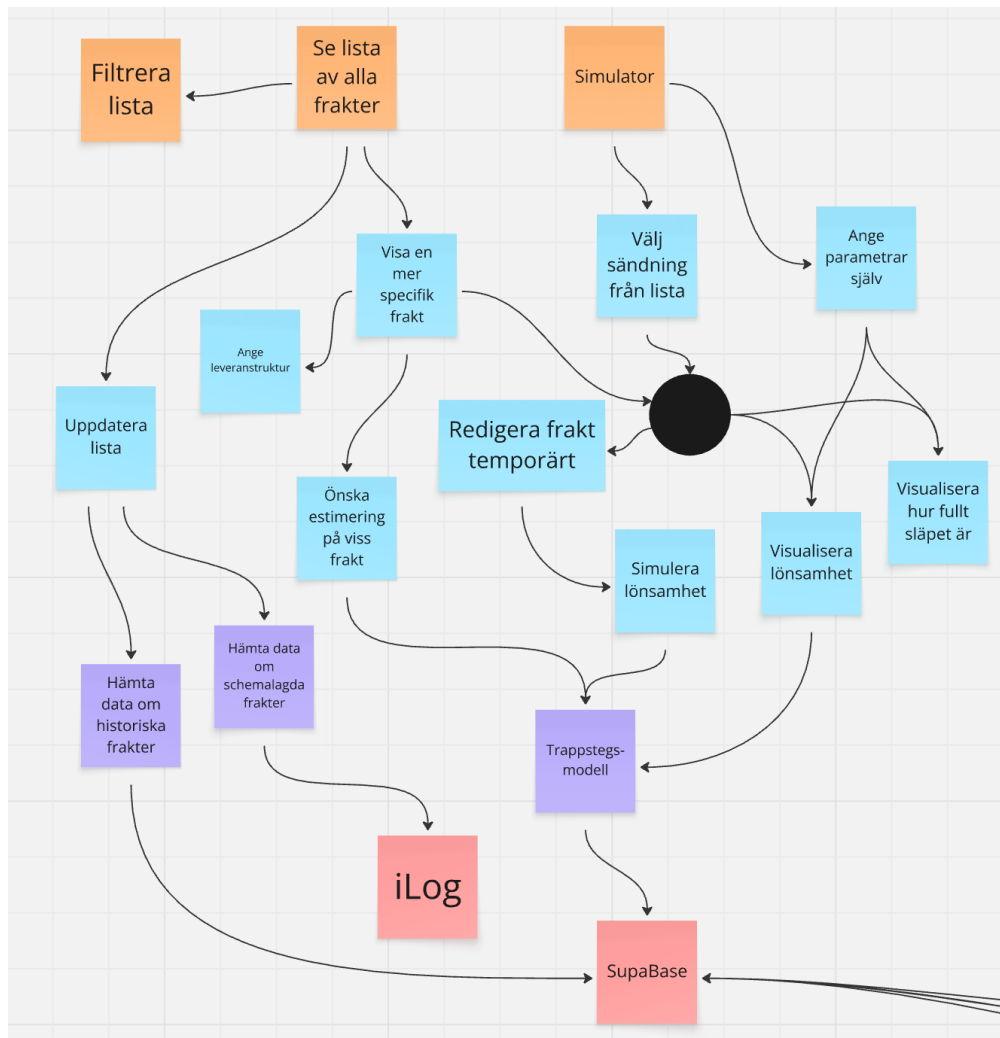
2.5 API och iLog

API:et och iLog som finns i figur 1 är system som detta projekt ej ska utveckla. Dessa system ska presenteras av kunden. iLog är ett stort system som många aktörer inom logistiksektorn använder runtom i hela Sverige, däribland kunden för detta projekt. Systemet är i sin helhet menat för att planera, schemalägga, beräkna och övervaka sändningar som skickas mellan geografiska punkter. Det API som är kopplat till iLog är gjort så att externa projekt skall kunna ta del av all data iLog har. Detta API är mycket extensivt med stor funktionalitet för hela iLogsystemet, och därför är det viktigt för detta projekt att enbart fokusera på de API-anrop som är av intresse.

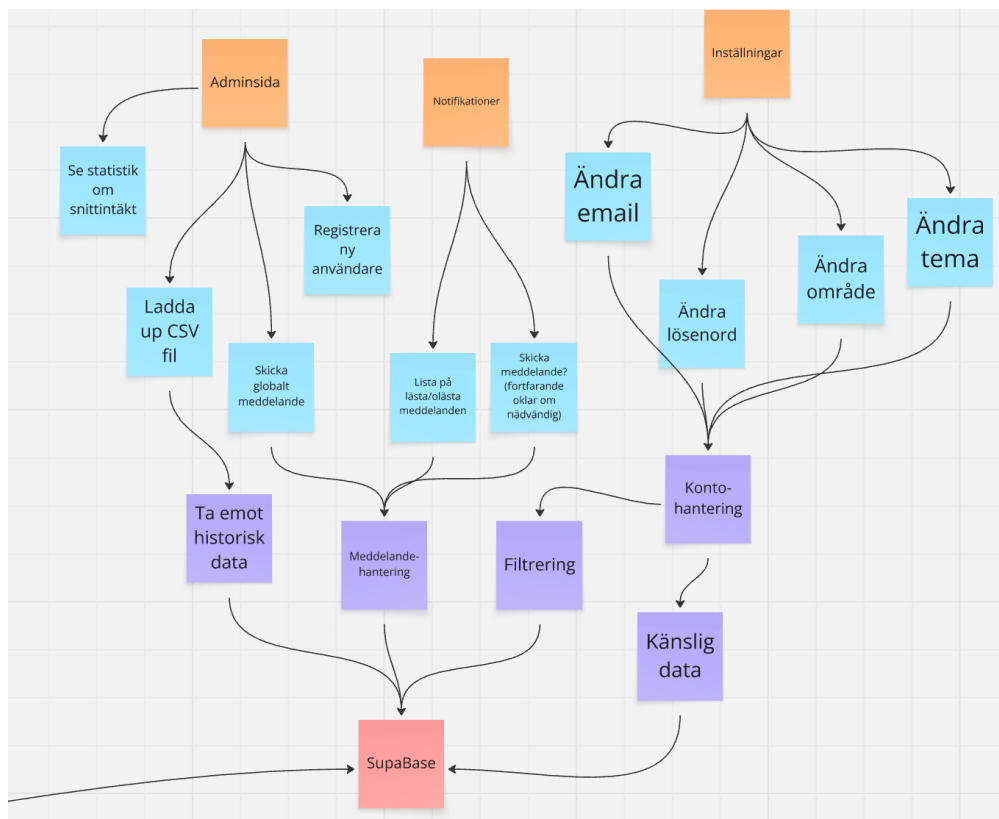
Detta API har både GET-anrop och PUSH-anrop. Eftersom detta projekt ej ska skriva någon data till iLog så är det viktigt att inga PUSH-anrop används, enbart GET-anrop skall användas. Av alla dessa API-rutter har vi identifierat tre som troligtvis kommer användas mest i detta projekt: En som hämtar data om sändningar på ett visst ekipage med en viss aktör, en är all planeringsdata för ett visst ekipage och en är detaljerad data om en enskild bokning.

3 SYSTEMANATOMI

I figurerna nedan visas ett komplett diagram över hur systemet i sin helhet ska se ut. Notera att denna iteration av systemanatomin är ganska stor så den har behövt delas upp i två figurer: figur 4 med det huvudsakliga syftet systemet löser och figur 5 med övrig funktionalitet kunden önskar. I båda figurerna har färgkodningen samma betydelse: orange är en tjänst, blått är användargränssnitt och *frontend*, lila är serverlogik och *backend* och rött är databaser. Denna systemanatomy har som utgångspunkt de krav som specificeras i dokumentet Kravspecifikation.



Figur 4: Systemanatomi för huvuddelen av projektet



Figur 5: Systemanatomi för övrig funktionalitet på kunds begäran